

DEVOPS TECHNICAL ASSIGNMENT

Engineering Deliverables & Architecture Documentation

Automated Infrastructure as Code (Terraform), CI/CD Deployment Pipelines (GitHub Actions), Observability, and High-Availability Multi-Tier Architecture on AWS.

ORGANIZATION

Octa Byte AI Pvt Ltd

AUTHOR / CANDIDATE

Tavkeer Shah

PRIMARY STACK

AWS ECS Fargate, RDS, Terraform

DATE & VERSION

August 2026 • v1.0.0

Architectural Approach & Technical Justifications

1. Executive Summary

This document details the engineering design decisions, trade-offs, and operational methodologies applied across the 3-tier containerized web application infrastructure and CI/CD pipelines for the DevOps Assignment.

2. Infrastructure Architecture & Design Choices

2.1 Compute Tier: AWS ECS Fargate vs. EC2 / EKS

- **Decision:** Deployed the containerized workloads using **AWS ECS Fargate** (Serverless Container Orchestration).
- **Rationale:**
- **Zero Node Management:** Eliminates OS patching, AMI maintenance, and EC2 capacity planning.
- **Task-Level Isolation:** Each container task gets dedicated kernel isolation and AWS VPC ENI (`awsvpc` network mode).
- **Cost Efficiency:** Supports **Fargate Spot** capacity providers (`FARGATE_SPOT`), reducing compute costs by up to 70% in non-production environments.
- **Simplicity vs. Kubernetes (EKS):** For a 3-tier web application, ECS avoids the high operational overhead, control-plane costs (\\$/month per EKS cluster), and complex CNI/CSI ingress controllers of Kubernetes.

2.2 Network Topology: Multi-AZ Defense-in-Depth

- **Decision:** Designed a 3-tier Multi-AZ VPC network layout (`public` , `private-app` , `private-db`).
- **Rationale:**
- **Public Subnets:** Host only the Application Load Balancer (ALB) and NAT Gateways. No application containers or databases reside here.
- **Private Application Subnets:** ECS tasks run here without public IP addresses, accessing the internet for registry pulls and third-party APIs via NAT Gateways.
- **Private Database Subnets:** Completely air-gapped with no route to the internet. Accessible strictly via port 5432 from the ECS Security Group.

2.3 Database Tier: Amazon RDS PostgreSQL 16

- **Decision:** Managed PostgreSQL 16 database with automated snapshots and storage autoscaling.
- **Rationale:**
- **Operational Reliability:** Automated backups, minor version maintenance, and point-in-time recovery (PITR).

- **Encryption:** AWS KMS encryption at rest (`gp3` storage) and SSL/TLS encrypted in-transit.
- **Storage Autoscaling:** Automatically increases storage capacity as database load grows from 20 GB up to 100 GB.

2.4 Secrets Management: AWS Secrets Manager

- **Decision:** Stored database credentials in AWS Secrets Manager rather than static environment variables.
 - **Rationale:**
 - **Zero Hardcoded Secrets:** DB passwords are dynamically generated via Terraform's `random_password` provider and referenced securely via IAM task execution roles.
 - **Auditability & Rotation:** Full AWS CloudTrail audit logs for secret access and native support for automated credential rotation.
-

3. CI/CD Pipeline Architecture (GitHub Actions)

3.1 Separation of Concerns: PR Validation vs. Continuous Delivery

- **PR Verification (`ci-pr.yml`):**
 - Runs backend test suite (22 unit & integration tests) and frontend Vite production build.
 - Dependency security audits (`npm audit --audit-level=high`).
 - Repository & filesystem vulnerability scanning via **Trivy**.
 - Blocks merging if tests fail or high-severity CVEs are detected.
 - **Continuous Delivery & Deployment (`cd-pipeline.yml`):**
 - Triggered upon merge to `main`.
 - Builds immutable Docker images tagged with Git SHA (`sha-${{ github.sha }}`) and `latest`.
 - Performs container image vulnerability scanning using **Trivy**.
 - Deploys automatically to **Staging** and runs automated smoke tests.
 - **Manual Approval Gate:** Employs GitHub Environments protection rules requiring explicit human approval before deploying to **Production**.
-

4. Observability & Centralized Logging

4.1 Application Metrics (RED Method)

- Embedded Prometheus client (`prom-client`) directly in the Express backend:
- **Rate:** `http_requests_total` partitioned by HTTP method, route, and status code.
- **Errors:** Real-time ratio of 5xx HTTP server errors to total requests.
- **Duration:** `http_request_duration_seconds` histogram providing \$p50\$, \$p95\$, and \$p99\$ response latency.

- **Database Telemetry:** `database_query_duration_seconds` histogram and `database_errors_total` .

4.2 Centralized Logging Strategy

- Application logs use Winston with structured JSON formatting (`timestamp` , `level` , `message` , `metadata`).
 - Integrated with AWS CloudWatch via ECS `awslogs` log driver with configurable retention periods (7 days staging / 30 days prod).
-

5. Summary of Best Practices

- **Infrastructure as Code (IaC):** 100% modular, parameterized, and validated Terraform.
- **Shift-Left Security:** Automated vulnerability and secret scanning at PR and container build stages.
- **Decoupled Lifecycle:** Terraform manages the infrastructure platform; CI/CD manages application images and rolling updates (`ignore_changes = [task_definition]`).
- **Resiliency & Auto-Healing:** ECS circuit breaker rollbacks and ALB multi-AZ health probes.

DevOps Assignment: Challenges Faced & Technical Resolutions

During the design, provisioning, and automation of this 3-tier production platform, several real-world DevOps engineering challenges were identified and resolved:

Challenge 1: The Infrastructure vs. Container Deployment "Chicken-and-Egg" Problem

The Challenge

When provisioning AWS ECS with Terraform for the first time, ECS task definitions and services require container image URIs before any task can start. However, the application Docker images had not yet been built or pushed to the private Amazon ECR repositories. Pointing directly to nonexistent image tags would cause Terraform apply to fail or get stuck waiting for service stability.

Technical Resolution

- **Bootstrap Image Defaults:** Configured Terraform task definitions to use lightweight, valid bootstrap image references (`public.ecr.aws/docker/library/node:20-alpine` and `public.ecr.aws/nginx/nginx:alpine`).
 - **Terraform Lifecycle Rules:** Added `lifecycle { ignore_changes = [task_definition] }` to both `aws_ecs_service` resources.
 - **Decoupled CI/CD:** When the GitHub Actions pipeline runs, it builds the actual code, tags the images with Git SHA (`sha-xyz`), pushes to ECR, and registers new task definition revisions without Terraform overwriting them on subsequent infrastructure runs.
-

Challenge 2: AWS Free Tier RDS Backup Retention Constraints

The Challenge

During initial RDS provisioning on AWS Free Tier accounts, the `aws_db_instance` resource failed with:

```
text
FreeTierRestrictionError: The specified backup retention period exceeds the maximum available to f
```

Default AWS Free Tier accounts enforce a maximum automated backup retention period of **1 day** (or `0` to disable), while standard enterprise defaults are typically 7 to 30 days.

Technical Resolution

- Parameterized `backup_retention_period` across the RDS module and environment configuration files:
 - Staging / Free Tier: Set `backup_retention_period = 1` in `[ environments/staging.tfvars ]` (file:///Users/tavkeersshah/developement/devops/assignment-devops-tavkeer/terraform/environments/staging.tfvars).
 - Production: Maintained `backup_retention_period = 1` in `[ environments/production.tfvars ]` (file:///Users/tavkeersshah/developement/devops/assignment-devops-tavkeer/terraform/environments/production.tfvars) for Free Tier compatibility.
 - This allowed smooth deployment on AWS Free Tier without sacrificing automated snapshot functionality.
-

Challenge 3: Metric Cardinality Explosion in Express Prometheus Exporter

The Challenge

When instrumenting the Express REST API with Prometheus RED metrics, routes with URL parameters (such as `GET /api/tasks/123` or `DELETE /api/tasks/456`) would generate unique Prometheus label values for every entity ID. In production with thousands of tasks, this leads to **metric cardinality explosion**, drastically increasing Prometheus memory usage and query latency.

Technical Resolution

- Built a custom route normalizer `normalizeRoute(req)` in `[ backend/src/metrics.js ]` (file:///Users/tavkeersshah/developement/devops/assignment-devops-tavkeer/backend/src/metrics.js):
 - Collapses dynamic numeric and UUID identifiers into parameterized route signatures (e.g. `/api/tasks/:id`).
 - Bounded metric cardinality to a constant set of static route labels regardless of database size.
-

Challenge 4: Security Hardening & Zero-Vulnerability Dependency Governance

The Challenge

Standard Node.js base images and initial frontend dependencies triggered `npm audit` warnings and potential container CVEs during vulnerability scanning.

Technical Resolution

- **Multi-Stage Non-Root Containers:** Configured backend `[ backend/Dockerfile ]` (file:///Users/tavkeersshah/developement/devops/assignment-devops-tavkeer/backend/Dockerfile) with a non-root `USER node` and minimal Alpine base image (`node:20-alpine`).
- **Dependency Resolution Overrides:** Upgraded Vite to `^6.2.0` and added explicit dependency resolution overrides for `esbuild` in `frontend/package.json`.

- **CI/CD Quality Gating:** Added automated **Trivy** container and repository scans in GitHub Actions to catch and block vulnerabilities before deployment. Both frontend and backend now report **0 vulnerabilities**.

Challenge 5: Least-Privilege Network Isolation Across Multi-Tier Components

The Challenge

Exposing database or internal service ports inadvertently in public or shared security groups violates least-privilege compliance.

Technical Resolution

- Chained security groups strictly by source security group ID rather than CIDR blocks:
- **ALB SG:** Ingress `80/443` from `0.0.0.0/0`.
- **ECS SG:** Ingress `80` (Frontend) and `5000` (Backend) allowed **strictly from ALB Security Group ID**.
- **RDS SG:** Ingress `5432` allowed **strictly from ECS Tasks Security Group ID**.
- **Private DB Subnets:** Isolated with no Internet Gateway or NAT routes.

Challenge 6: Nginx Dynamic DNS Resolution in ECS Fargate

The Challenge

In local Docker Compose, Nginx can resolve static container names (`http://backend:5000`) at boot. However, in AWS ECS Fargate, the Application Load Balancer routes `/api/*` directly to the backend. A static `proxy_pass http://backend:5000` caused Nginx to crash at startup with `[emerg] host not found in upstream "backend"`, causing ECS tasks to restart in a loop.

Technical Resolution

- Implemented dynamic variable-based DNS resolution in [`frontend/nginx.conf`]
(`file:///Users/tavkeersshah/development/devops/assignment-devops-tavkeer/frontend/nginx.conf`):

```
nginx
resolver 127.0.0.11 ipv6=off valid=30s;

set $backend_upstream http://backend:5000;

proxy_pass $backend_upstream;
```

- Nginx evaluates DNS on request rather than crashing at boot, allowing immediate container readiness and instant ALB health probe passes.

Challenge 7: AWS RDS PostgreSQL SSL/TLS Enforcement

The Challenge

AWS RDS PostgreSQL instances mandate SSL/TLS encryption for incoming client connections (`hostssl`), rejecting unencrypted pool requests with `no pg_hba.conf entry ... no encryption` .

Technical Resolution

- Configured `pg` connection pool in [`backend/src/db.js`] (file:///Users/tavkeersshah/developement/devops/assignment-devops-tavkeer/backend/src/db.js) to dynamically detect cloud database hosts and enforce SSL encryption (`ssl: { rejectUnauthorized: false }`).

Challenge 8: AWS 32-Character Maximum Limit on ALB Target Group Names

The Challenge

AWS Elastic Load Balancing enforces a strict 32-character maximum limit on Target Group names. When `environment = "production"` (10 characters), the default name `${var.environment}-${var.project_name}-fe-tg` exceeded 32 characters, and naive string slicing left trailing hyphens (`-`) which AWS rejects.

Technical Resolution

- Implemented environment name prefixing in [`terraform/modules/alb/main.tf`] (file:///Users/tavkeersshah/developement/devops/assignment-devops-tavkeer/terraform/modules/alb/main.tf):

```
hcl
locals {

    name_prefix = var.environment == "production" ? "prod-${var.project_name}" : "${var.environment}-${var.project_name}"

}
```

- Guaranteed all target groups stay under 28 characters with zero trailing hyphens.

3. Infrastructure Cost Analysis & Optimization

Comprehensive cost projection calculated for AWS `us-east-1` (North Virginia) based on the provisioned Terraform resources:

AWS Resource	Configuration	Daily Cost	Monthly Cost (30 Days)	Optimization Strategy
AWS NAT Gateway	1 Single-AZ Gateway	~\$1.10 / day	\$32.85	Single NAT Gateway mode enabled
Application Load Balancer	1 Internet-Facing ALB	~\$0.60 / day	\$18.00	Shared ALB with path-based routing (/api/*)
AWS ECS Fargate	4 tasks (2 Frontend + 2 Backend)	~\$0.36 / day	\$10.80	Fargate Spot enabled (~70% discount)
Amazon RDS PostgreSQL 16	db.t4g.micro (Single-AZ, 20GB gp3)	~\$0.00 (Free Tier)	\$0.00 (Free Tier) (\$13.90 standard)	Free Tier eligible micro instance & storage
AWS Secrets Manager	1 Database Secret	~\$0.01 / day	\$0.45	Consolidated dynamic secret
CloudWatch Logs & Metrics	7-day retention + 1 Dashboard	~\$0.02 / day	\$0.50	Explicit retention period limits storage bloat
Amazon ECR	2 Repositories (~200MB)	~\$0.00 / day	\$0.05	Lifecycle policies purge untagged images
AWS IPv4 Address Fee	1 Public IPv4 (ALB/NAT)	~\$0.12 / day	\$3.65	Standard AWS IPv4 charge
TOTAL (Staging Environment)		~\$2.20 / day	~\$66.00 / month	(Or ~\$52/mo if inside Free Tier window)

Cost Optimization Highlights:

- **Fargate Spot Computing:** Saved ~70% compute costs compared to standard on-demand Fargate.
- **Single NAT Gateway for Staging:** Reduced monthly NAT Gateway costs from \$65/mo down to \$32.85/mo.
- **ECR Lifecycle Automation:** Automatically purges untagged layers older than 7 days and limits tagged releases to the last 30 versions.

- **Log Retention Policies:** Explicit 7-day retention rule in CloudWatch stops continuous log storage accumulation.